Instruction Tuning and Direct Preference Optimization on Qwen2.5-0.5B

Programming Assignment 2 — Final Report

Name, Roll number

Repository: https://github.com/<user>/small-model-alignment

Abstract

We post-train `Qwen/Qwen2.5-0.5B` with a from-scratch supervised fine-tuning (SFT) loop on 30k UltraChat examples, then run two from-scratch DPO experiments from the same SFT checkpoint: one on 15k single-turn HH-RLHF pairs and one on 15k UltraFeedback pairs. A third track tunes `SmolLM2-360M-Instruct` with TRL's `DPOTrainer` and LoRA toward a Gordon-Ramsay-style persona. All models are compared on fixed prompt sets with blinded, order-swapped pairwise judgments from `google/gemini-3.6-flash` through Kaggle's model proxy. SFT lowers held-out response NLL from 1.396 to 1.331 and makes the model stop properly (65% of base-model answers hit the 512-token cap). UF-DPO reaches 0.775 validation reward accuracy. For HH, the guide learning rate (10^{-6}) barely changes refusal behavior, so we sweep $\{1, 3, 10\} \times 10^{-6}$ and select 3×10^{-6} by validation reward accuracy (0.654). It raises refusal and redirection on harmful prompts, while 10^{-5} over-optimizes into incoherent over-refusal. Judged against SFT, UF-DPO wins xx% of AlpacaEval comparisons and HH-DPO wins xx% of helpful/harmless comparisons.

1 Repository, models, and data

Code. The training math (batching, masking, response log-probabilities, SFT and DPO losses) lives in `sma/core.py`, and `submission_adapter.py` wraps it for the private grader. The loops are `sma/train_sft.py` and `sma/train_dpo.py`; neither uses HF `Trainer` or TRL. The 7 public tests and 15 additional edge-case tests pass (truncation that removes EOS, `pad_token_id=eos_token_id`, missing pad/EOS tokens, row order, brute-force log-prob checks, gradient-accumulation equivalence).

Prompt format. The same template is used for SFT, both DPO runs, and evaluation: Qwen ChatML with the system message "You are a helpful assistant." (`sma/prompts.py`). The response is followed by the tokenizer EOS `<|endoftext|>`, and generation stops on `<|im_end|>` or `<|endoftext|>`. Prompt and response are tokenized separately and concatenated, EOS is appended *before* truncation to 512 tokens, rows are right-padded, and all masks are derived from true row lengths (important because Qwen's pad id equals its EOS id).

Seeds and decontamination. Seed 42 is used for every shuffle, the evaluation subset, data order, and A/B order. Training prompts that exactly match (after whitespace and case normalization) an evaluation instruction are removed.

Table 1: Data actually used. Counts are from `outputs/data/data_stats.json`.

Track	Source	Filtering	Train / val
SFT	UltraChat-200k (`HuggingFaceH4/ultrachat_200k`)	first user/assistant turn; prompt ≤256 tok; prompt+response+EOS ≤512 tok (30,000 kept of 59,174 scanned)	30,000 / 1,000
HH-DPO	`Anthropic/hh-rlhf`, harmless-base + helpful-base	single-turn only (24,971 of 86,372); chosen ≠ rejected; prompt ≤256 tok; 50/50 harmless/helpful	15,000 / 1,000
UF-DPO	UltraFeedback (`HuggingFaceH4/ultrafeedback_binarized`)	single-turn; score margin ≥1 (drops 18,447 near-ties); prompt ≤256 tok (33,474 eligible)	15,000 / 1,000
LoRA-DPO	provided `ramsay_style_preferences.json`	seeded 90/10 split	198 / 22

Evaluation sets: AlpacaEval subset of 300 of 805 prompts drawn with `random.Random(42).sample` (`eval_sets/alpaca_eval_300_seed42.json`); the provided HH set (300 prompts, 10 categories × 30); our persona set of 40 prompts (10 each: cooking, coding, general help, safety-sensitive).

2 Experimental configuration

For each pair (x, y_w, y_l) we minimize the DPO objective, with response-only summed log-probabilities:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\Big(\beta\big[(\log \pi_\theta(y_w \mid x) - \log \pi_\theta(y_l \mid x)) - (\log \pi_{\text{ref}}(y_w \mid x) - \log \pi_{\text{ref}}(y_l \mid x))\big]\Big), \tag{1}$$

$$r_w = \beta\big(\log \pi_\theta(y_w \mid x) - \log \pi_{\text{ref}}(y_w \mid x)\big), \qquad r_l = \beta\big(\log \pi_\theta(y_l \mid x) - \log \pi_{\text{ref}}(y_l \mid x)\big).$$

Reward accuracy is $\Pr[r_w > r_l]$ (strict) and the margin is $\mathbb{E}[r_w - r_l]$. The SFT loss is the *token mean* of response NLL. With gradient accumulation it is normalized by the response-token count of the whole accumulation window, so the accumulated gradient equals the gradient of the token mean over the full effective batch.

Table 2: Values used for the submitted checkpoints (one H100 80GB on Modal; seed 42 throughout).

	SFT	HH-DPO	UF-DPO	LoRA-DPO
Model	Qwen2.5-0.5B	SFT best	SFT best	SmolLM2-360M-Instruct
Trainable	full (494M)	full	full	LoRA r=16, α=32, drop. 0.05; q,k,v,o,gate,up,down (8.68M, 2.34%)
Optimizer	AdamW (0.9, 0.999)	AdamW	AdamW	AdamW (TRL)
Peak LR, schedule	2×10^{-5}, cosine, 3% warmup	3×10^{-6} constant (sweep $\{1,3,10\}\times10^{-6}$)	10^{-6} constant	sweep $\{5\times10^{-5}, 10^{-4}, 2\times10^{-4}\}$, cosine, 10% warmup
Weight decay / clip	0.1 (not on norms/emb.) / 1.0	0 / 1.0	0 / 1.0	0 / 1.0
Micro × accum = eff.	$16 \times 2 = 32$	$4 \times 8 = 32$ pairs	$4 \times 8 = 32$ pairs	$4 \times 2 = 8$
Precision	bf16 autocast, FP32 weights	same	same	bf16
Max len / prompt	512 / 256	512 / 256	512 / 256	512 / 256
Epochs (steps)	1 (938)	1 (469)	1 (469)	3 (75) / 3 (75) / 5 (125)
β, reference	—	0.1, frozen `sft/best`	0.1, frozen `sft/best`	0.1, adapter disabled
Selection	min val loss (step 900)	max val reward acc (step 400)	max val reward acc (step 400)	final adapter; run chosen by judge (§5.1)
Wall-clock	6.0 min	7.3 min (each run)	12.4 min	∼1 min

Decoding (identical within every comparison). Greedy (`do_sample=False`, so temperature and top-p are unused), `max_new_tokens`=512, `repetition_penalty`=1.1, no n-gram blocking, bf16, left-padded batches. The Qwen models use the ChatML template above; the SmolLM2 models use their own chat template. **Judge.** `google/gemini-3.6-flash` at temperature 0, 5 pairs per request, ≥5 s between attempts, ≤3 retries, both A/B orders.

3 Parameter choices and deviations

Table 3: Deviations from the guide. R = resource-driven, P = performance-driven, D = design or measurement choice.

Item	Guide	Ours	Type	Reason and observed effect
Hardware	T4×2, fp16+scaler	H100, bf16	R	bf16 has fp32 range, so no loss scaling is needed and no steps are skipped; FP32 master weights unchanged. The fp16 path is kept (`-precision fp16`).
SFT micro-batch	2×16	16×2	R	Same effective batch; window-level token normalization makes the gradient identical (unit-tested).
DPO micro-batch	1×32	4×8	R	Same 32 pairs per step; per-micro-batch means are reweighted.
DPO optimizer	RMSprop/AdamW	AdamW	D	RMSprop has no bias correction: its second moment starts at 0, so early steps are about 10× lr without warmup. AdamW is stable at a fixed 10^{-6}.
Reference model	fp16, 2nd GPU	FP32 + same autocast	D	Policy and reference are bit-identical at step 0: we measured loss $= 0.6931 = \log 2$ and accuracy $= 0$, margin $= 0$ on both tracks, so diagnostics measure learning rather than precision noise.
SFT length filter	—	fit in 512	P	Every target ends in EOS. SFT outputs: 0% truncated on the sanity set vs. 65.3% for the base model on AlpacaEval.
UF pair filter	—	margin ≥ 1	P	Removes ties and near-ties (30% of pairs), which carry little preference signal.
HH-DPO LR	10^{-6}	3×10^{-6}	P	At 10^{-6} the model still complies with 100% of harmful/cyber/doxxing prompts (Table 4). Of $\{1, 3, 10\} \times 10^{-6}$ we select by validation reward accuracy, not by judge results. The guide run is still reported.
LoRA LR / epochs	—	3-point sweep	P	The first run (5×10^{-5}) saturated validation accuracy but barely changed greedy outputs, so we swept LR and epochs and compared runs by judged style *and* quality (§5.1).

4 Training behavior

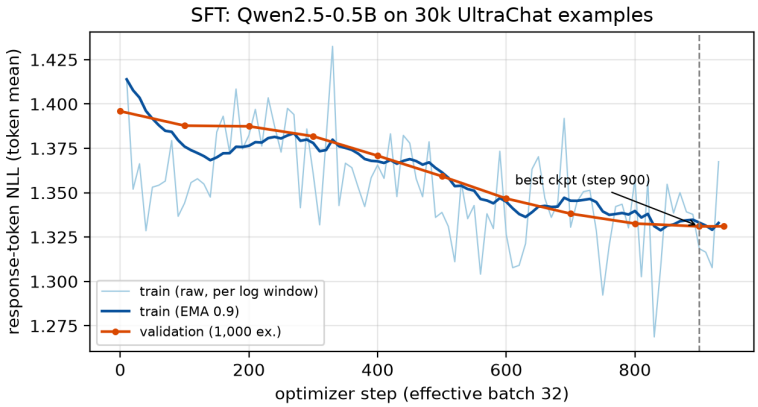


Figure 1: SFT response-token NLL. Train loss per 10-step window (raw and EMA 0.9); validation on 1,000 held-out examples every 100 steps. The dashed line marks the selected checkpoint (step 900).

SFT. Validation NLL falls monotonically from 1.396 (base) to 1.331 (step 900; step 938 is equal to 4 d.p.). It is nearly flat at peak LR (1.388 at steps 100–200) and drops mostly during cosine decay, with no sign of overfitting. The modest change is expected: the base model already models UltraChat-style text, and SFT mainly teaches the turn format and when to stop.

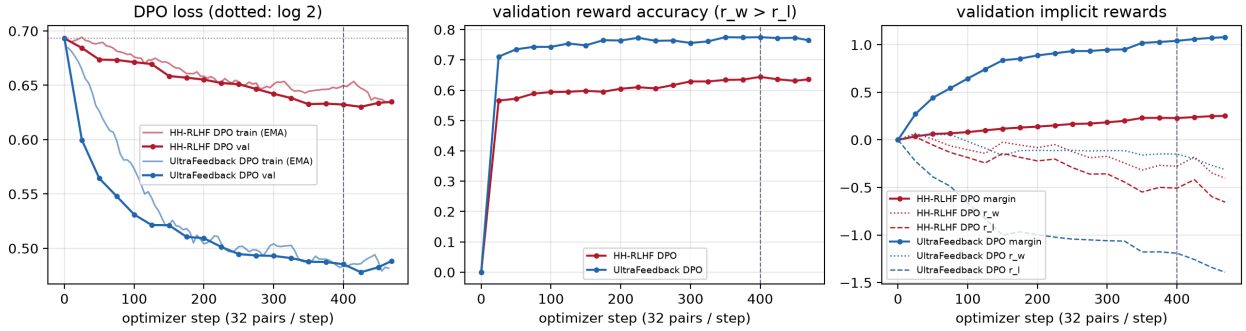


Figure 2: From-scratch DPO. Left: train loss (EMA 0.9) and validation loss (dotted line: $\log 2$). Middle: validation reward accuracy. Right: validation margin and implicit rewards r_w (dotted), r_l (dashed). Vertical lines mark the selected checkpoints (step 400 for both runs). Evaluation every 25 steps on 1,000 pairs.

DPO. Both runs start at exactly $\log 2$ with zero accuracy and margin (all rewards tied), as Eq. 1 predicts for identical policy and reference. UF learns much faster and further: accuracy 0.711 after 25 steps and 0.775 at step 400 (margin 1.04). HH reaches 0.644 (margin 0.23) and climbs slowly throughout, consistent with HH's noisier labels. In both runs *both* implicit rewards become negative, which means the chosen log-likelihood also falls:

Run (best step)	val acc	margin	r_w	r_l	$\log \pi(y_w)$ / $\log \pi(y_l)$: init → best
HH-DPO (400)	0.644	0.230	−0.278	−0.507	−119.5 → −122.3 / −115.1 → −120.2
UF-DPO (400)	0.775	1.042	−0.150	−1.191	−319.7 → −321.2 / −276.5 → −288.4

DPO therefore widens the gap mainly by pushing rejected responses down (the "likelihood displacement" effect). Accuracy saturates after about 350 steps in both runs, so the best-accuracy checkpoint (step 400) and the final checkpoint differ little.

HH learning-rate sweep. Table 4 and Fig. 3 show that validation accuracy is nearly flat across a 10× LR range, while the margin and the behavior change a lot. The refusal columns use a keyword heuristic (refusal, decline, or redirection to professional help in the first 400 characters) on the 30 prompts per category of HH-eval. They are a measurement aid; the judge results in §5 are the actual evaluation. At 10^{-5} the chosen log-likelihood collapses (−119.5 → −138.4), validation loss rises above the other runs, and greedy outputs

Table 4: HH-DPO learning-rate sweep (one epoch, β=0.1, all from `sft/best`). Columns 2–5 are validation diagnostics at the best step (400; 469 for 10^{-5}); the last five columns are measured on HH-eval generations.

	Validation (1,000 pairs)					HH-eval generations: refuse/redirect rate				
LR	acc	margin	loss	$\log \pi(y_w)$	tokens (capped)	harmful	cyber	privacy	hate	benign
SFT (reference)	—	—	0.693	−119.5	177 (0.7%)	0%	0%	0%	3%	0%
10^{-6} (guide)	0.644	0.230	0.632	−122.3	152 (0.7%)	0%	0%	0%	0%	0%
3×10^{-6} (selected)	**0.654**	0.420	**0.631**	−122.5	137 (0.7%)	10%	3%	20%	23%	0%
10^{-5}	0.649	0.723	0.695	−138.4	405 (69%)	90%	100%	90%	100%	23%

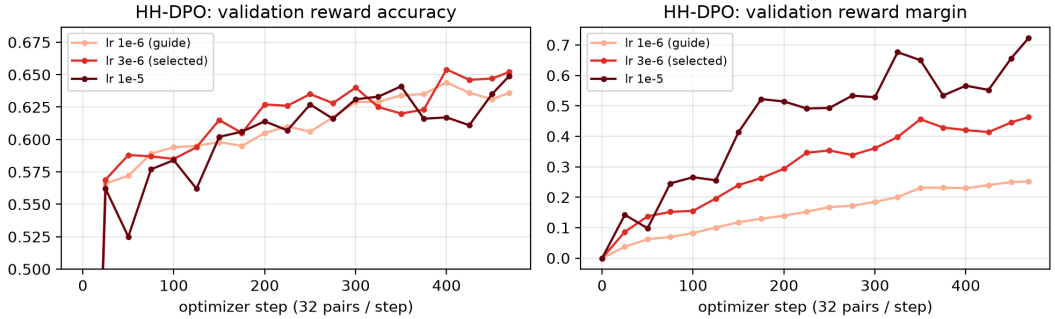


Figure 3: HH-DPO LR sweep: validation reward accuracy (left) and margin (right), evaluated every 25 steps.

become long, repetitive refusals (69% hit the token cap). We therefore select 3×10^{-6}, which has the best validation accuracy *and* loss.

LoRA-DPO. On the 22 held-out persona pairs, reward accuracy reaches 1.0 within about 10 steps in every run, so it carries no information. Figure 4 shows the margin and log-probabilities across the sweep. At 5×10^{-5} DPO raises the chosen log-likelihood ($-174 \to -163$) and mildly lowers the rejected one. At 10^{-4} the rejected likelihood falls sharply ($-126 \to -171$) while the chosen stays above its initial value. At 2×10^{-4} for 5 epochs the chosen likelihood *also* collapses ($-174 \to -195$) and the margin saturates at 10.3, a textbook sign of over-optimization.

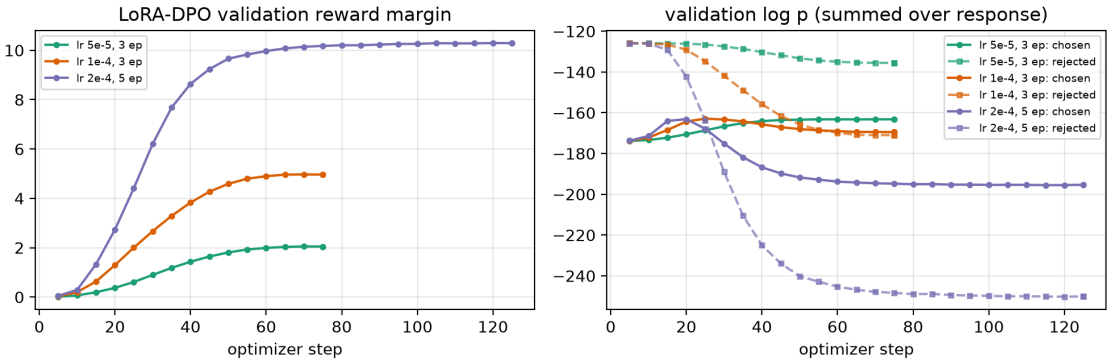


Figure 4: LoRA-DPO sweep on SmolLM2-360M-Instruct. Left: validation margin. Right: summed validation log-probabilities of the chosen (solid) and rejected (dashed) responses.

5 DPO versus SFT results

Every comparison uses the same prompts, the same decoding settings, and the same judge procedure. The judge sees only the prompt, the per-prompt guidance (HH set), and "Response A/B". Each pair is judged in both orders; a win counts only if both orders agree, otherwise the pair is a tie (MT-Bench convention). Unresolved items after 3 retries are reported as failures and excluded from the rates.

Response length (measured, no judge). Mean generated tokens, with the fraction hitting the 512-token cap in parentheses:

Table 5: Main results (candidate vs. SFT). CI: bootstrap 95% interval of the ties-as-half score.

Comparison (set)	n	W	T	L	win	win excl. T	ties= $\frac{1}{2}$ (95% CI)	fail/unk.	pos. consist.
UF-DPO vs SFT (AlpacaEval)	300								
HH-DPO 3×10^{-6} vs SFT (HH-eval)	300								
HH-DPO 10^{-6} (guide) vs SFT (HH-eval)	300								
Additional analysis (not headline numbers)									
HH-DPO 10^{-5} vs SFT (HH-eval)	300								
HH-DPO 3×10^{-6} vs SFT (AlpacaEval)	300								
UF-DPO vs SFT (HH-eval)	300								
SFT vs base (AlpacaEval)	300								

Eval set	Base	SFT	HH-DPO 10^{-6}	HH-DPO 3×10^{-6}	UF-DPO
AlpacaEval-300	427 (65.3%)	185 (2.3%)	180 (3.0%)	179 (6.3%)	294 (9.7%)
HH-eval-300	375 (49.0%)	177 (0.7%)	152 (0.7%)	137 (0.7%)	280 (2.7%)

SFT fixes termination. UF-DPO lengthens answers by about 60% on both sets; HH-DPO slightly shortens them, most on HH-eval.

5.1 Persona track (reported separately)

Table 6: LoRA-DPO runs vs. SmolLM2-360M-Instruct on the 40-prompt persona set (both orders judged). The judge gives two separate decisions per pair: persona *style* and *quality* (usefulness, correctness, safety; persona ignored).

Run	val margin	$\log\pi(y_w)$ / $\log\pi(y_l)$	tokens (capped)	Style W/T/L	Quality W/T/L
Base SmolLM2	—	−174.0 / −125.8	233 (0%)	—	—
lr 5×10^{-5}, 3 ep	2.05	−163.2 / −135.5	243 (5%)		
lr 10^{-4}, 3 ep	4.96	−169.5 / −171.0	198 (7.5%)		
lr 2×10^{-4}, 5 ep	10.29	−195.3 / −250.1	409 (70%)		

Qualitative (raw-chicken prompt). The base model and the 5×10^{-5} run both say the chicken can still be cooked (unsafe), the latter with only mild flavor ("Hmmm..."). The 10^{-4} run is in persona *and* correct ("You can't even THINK about cooking that raw chicken ... put it in the fridge ... you'll get sick!"). The 2×10^{-4} run degenerates into loops ("You're a monster!" repeated until the token cap).

6 Interpretation and error analysis

To fill from the judged files (`*_judged_final.jsonl`, per-category tables):

- **Measured:** which model wins on each set, by how much, and whether the CI excludes 50%.
- **UF-DPO:** length change (mean characters/tokens vs SFT) and whether wins track length; per-source breakdown.
- **HH-DPO:** per-category wins (harmful, cyber, privacy, self-harm) vs. possible over-refusal on benign and calibration prompts. Short, noisy HH chosen responses and the small KL budget (β=0.1, 469 steps at 10^{-6}).
- **Examples:** 2–3 improvements and 2–3 regressions per track.
- **Observed in sanity checks:** SFT produces fluent but sometimes hallucinated content (e.g. listing living bear species, and even tigers, as "extinct" bears), which DPO on 15k pairs is unlikely to fix.
- **Persona:** style strength is controlled by the effective KL budget (LR × steps at fixed β). Too little leaves greedy outputs unchanged; too much triggers likelihood collapse and repetition (Table 6, Fig. 4).

7 Limitations and reproducibility

- Single seed per run. With n=300 the win-rate CI is roughly ±5–6 points. One LLM judge, whose biases remain despite order swapping and the explicit anti-length instruction. The persona evaluation set (40) and

its validation split (22) are small.

- Greedy bf16 decoding in length-sorted batches can produce small batch-composition effects. Settings are identical across the models in each comparison.
- **Reproduce:** `modal run modal_app.py` (or `notebooks/kaggle_t4_pipeline.ipynb`), then `notebooks/kaggle_judge.ipynb`. Key artifacts:

Configs	`configs/{sft,dpo_hh,dpo_uf,lora_dpo}.yaml`
Training logs	`outputs/{sft,dpo_hh,dpo_uf}/{run_config,summary}.json`, `metrics.jsonl`
LoRA runs	`outputs/lora_dpo*/{summary,log_history,split}.json`
Data statistics	`outputs/data/data_stats.json`
Generations	`outputs/generations/{alpaca,hh,persona}/<model>.json` (+ `.meta.json`)
Pair files	`outputs/eval/<comparison>_pairs.jsonl`
Judgments	`outputs/judged/<comparison>_judged{,_failures,_summary,_final}.*`

References

[1] R. Rafailov et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.

[2] L. Ouyang et al. Training language models to follow instructions with human feedback. NeurIPS 2022.

[3] Y. Bai et al. Training a Helpful and Harmless Assistant with RLHF. 2022.

[4] Y. Dubois et al. Length-Controlled AlpacaEval. 2024.

[5] L. Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS 2023.

[6] Hugging Face Transformers, TRL, and PEFT documentation; Kaggle Benchmarks documentation.